Journal of Network and Computer Applications 233 (2025) 104069

Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca

SAT-Net: A staggered attention network using graph neural networks for encrypted traffic classification

Zhiyuan Li, Hongyi Zhao, Jingyu Zhao, Yuqi Jiang, Fanliang Bu *

School of Information Network Security, People's Public Security University of China, Beijing, 100045, China

ARTICLE INFO

Handling editor: M. Atiquzzaman

Keywords:
Deep learning
Encrypted traffic classification
Graph neural networks
Attention mechanism

ABSTRACT

With the increasing complexity of network protocol traffic in the modern network environment, the task of traffic classification is facing significant challenges. Existing methods lack research on the characteristics of traffic byte data and suffer from insufficient model generalization, leading to decreased classification accuracy. In response, we propose a method for encrypted traffic classification based on a Staggered Attention Network using Graph Neural Networks (SAT-Net), which takes into consideration both computer network topology and user interaction processes. Firstly, we design a Packet Byte Graph (PBG) to efficiently capture the byte features of flow and their relationships, thereby transforming the encrypted traffic classification problem into a graph classification problem. Secondly, we meticulously construct a GNN-based PBG learner, where the feature remapping layer and staggered attention layer are respectively used for feature propagation and fusion, enhancing the robustness of the model. Experiments on multiple different types of encrypted traffic datasets demonstrate that SAT-Net outperforms various advanced methods in identifying VPN traffic, Tor traffic, and malicious traffic, showing strong generalization capability.

1. Introduction

With the rapid development of computer communication and Internet of Things (IoT) technologies, the demand for data security and privacy protection has reached a new level. According to the Google Transparency Report (Google) (the portion of HTTPS encryption on the web), as of February 11, 2024, 96% of all Google products and services have implemented encryption. Therefore, in the unstoppable trend of network traffic encryption, effectively classifying encrypted traffic into specific categories is a key issue for achieving high-quality network security management and optimizing network resources (e.g., QoS control, traffic billing, and intrusion detection) (Tao et al., 2023). The process of network traffic classification typically involves capturing traffic data from monitored network nodes and then labeling them according to user requirements (protocols, service types). However, as traffic encryption technologies continue to evolve (such as IPsec, TLS, SSL, and SRTP), achieving fine-grained encrypted traffic classification faces many unprecedented challenges.

Due to concerns about user privacy and security, numerous technologies have emerged for secure communication and remote access, such as Virtual Private Networks (VPN), Network Address Translation (NAT), and The Onion Routing (Tor). This directly results in a significant decrease in the accuracy of traditional methods based on port and deep packet/flow inspection (Yaxuan et al., 2009) (Tomasz et al., 2015) (Michael et al., 2014). The reasons for this are as follows: (1) Many applications no longer have queryable port numbers (because of the use of dynamic port numbers) (Michelle et al., 2011) and employ port obfuscation techniques to bypass firewall detection. (2) The proportion of encrypted fields in packet continues to increase, making the use of payload unreliable. As the proportion of encrypted traffic gradually increases, machine learning and deep learning-based methods for network traffic analysis have become mainstream. Researchers have found that encrypted traffic of different types (e.g., protocol type, service type, and application type) exhibits different statistical characteristics. Thus, they attempt to combine machine learning algorithms (e.g., C4.5 decision tree, SVM, RF) with manually designed statistical features (e.g., time-related features) for encrypted traffic classification (Gerard et al., 2016) (Arash Habibi et al., 2017). However, this approach has certain drawbacks. On the one hand, it is difficult to ensure that the selected traffic statistical features are stable and reliable. For example, the statistical characteristics of Elephant flows and Mice flows are significantly different in their respective communication patterns (Theophilus et al.,

* Corresponding author.
E-mail address: bufanliang@sina.com (F. Bu).

https://doi.org/10.1016/j.jnca.2024.104069
Received 21 March 2024; Received in revised form 21 October 2024; Accepted 13 November 2024
Available online 15 November 2024
1084-8045/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

2010). On the other hand, this method heavily relies on expert experience and consumes a considerable amount of manual effort during the feature engineering stage.

To address the aforementioned challenges, researchers have turned their attention to deep learning methods. By simply inputting raw network traffic data into a well-designed neural network model, the model can automatically extract features for traffic classification (Wei et al., 2017a; Haipeng et al., 2022; Chang et al., 2019; Xinjie et al., 2022; Peng et al., 2023; Giuseppe et al., 2021; Xin et al., 2020). Due to its linear features such as translation invariance and compositionality, CNN is naturally suitable for deep computation and inference of Euclidean data, such as natural language processing and image processing tasks (Ma et al., 2022). Therefore, we believe that the potential reason for the effectiveness of deep learning methods in encrypted traffic classification lies in the representation of traffic flow data (Chang et al., 2019) (Xin et al., 2020) (Chang et al., 2018), which inevitably brings network traffic into Euclidean space. However, this approach has two drawbacks: first, it cannot fully explore the nonlinear implicit features of network traffic, and second, training deep neural network models consumes a significant amount of time.

Recently, researchers have become increasingly interested in non-Euclidean data, which is widely present in many fields (Zonghan et al., 2021) (such as social networks, chemical molecular structures, and neuroscience), and have attempted to model and analyze network traffic according to non-Euclidean structures (Chang et al., 2018; Chen et al., 2020; Ting-Li et al., 2021, 2023; Minghao et al., 2022; Bo et al., 2021; Meng et al., 2021). Therefore, starting from the topology of computer networks, we observe that the features of network traffic data naturally reside in the non-Euclidean domain. Representing network encrypted traffic data as graph data can express its potential hidden patterns. As shown in Fig. 1, the arrangement of elements in the two data domains exhibits significant differences. Specifically, treating bytes/packets/session flows in encrypted traffic as nodes in the graph and constructing edges using relationships between corresponding entities, embedding the traffic graph into a model based on graph neural networks, and then completing downstream tasks according to user requirements. Existing GNN-based methods primarily use packets as graph nodes to extract flow-level feature information for encrypted traffic classification. In summary, we believe that focusing on the intrinsic characteristics of traffic flows and achieving fine-grained feature extraction can, to some extent, alleviate the issue of feature instability caused by fluctuations in the network environment. Therefore, selecting a suitable, stable, and reliable representation method for encrypted traffic is a crucial step towards efficient encrypted traffic classification and enhancing the generalization capability of the model.

Inspired by the observations of geometric deep learning, in this paper, we propose SAT-Net (Staggered Attention Network), a novel GNN-based method for encrypted traffic classification. Specifically, SAT-Net comprises four modules: Graph Embedding Layer, Feature Remapping Layer, Staggered Attention Layer, and classification layer, with detailed graph construction and model design presented in Section4. Firstly, SAT-Net addresses the challenge of converting traffic flows into graph. Specifically, we transform the original byte sequence of data packets into an undirected graph structure, namely the Packet Byte Graph (PBG), where bytes serve as vertices in the graph, and the associations between bytes are represented by edges. This graph structure allows for capturing the complex relationships between bytes, laying the foundation for subsequent feature learning. Next, we perform embedding learning on the PBG through the GraphSAGE algorithm, obtaining vector representations in the feature space. Subsequently, these vectors are subjected to feature remapping by a carefully designed MLP (Multi-Layer Perceptron), thereby extracting richer and more expressive feature representations. In particular, we introduce a multi-head attention mechanism in the Staggered Attention Layer, which fuses parallel PBG feature vectors to capture global context and key features, thereby highlighting important information in the packet byte graph of encrypted traffic. Ultimately, these fused features are input into the classification layer, generating the final classification labels. With this architecture, SAT-Net effectively captures key patterns and features in encrypted traffic, significantly improving the accuracy and robustness of classification. Especially through the combination of PBG and the Staggered Attention Layer, SAT-Net demonstrates strong performance advantages in encrypted traffic classification tasks.

In summary, the main contributions of our paper are summarized as follows:

- We propose a GNN-based encrypted traffic classification method, named SAT-Net, which transforms the network traffic classification problem into a graph classification problem. It handles graph feature vectors in a multi-task learning manner, enabling high-precision packet-level encrypted traffic classification.
- To achieve fine-grained feature extraction, we design a dynamic sliding window size selection strategy to obtain optimal representations of packet byte graph of traffic flow, thereby improving the model's generalization.
- To obtain vector representations of entire packets, we propose a feature fusion method based on staggered attention, allowing it to capture important features from different graphs and enhance the model's performance.
- We evaluate the performance of our proposed model through a series of experiments on four public datasets and a self-collected dataset. Experimental results demonstrate that, compared to baselines, SAT-Net can accurately identify encrypted traffic generated by different apps with high precision.

The rest of this paper is organized as follows. In Section 2, we present an overview of the related research on encrypted traffic classification according to various methodologies. Section 3 serves as a preliminary section, setting the stage for the subsequent discussion. Section 4 provides a detailed description of our proposed SAT-Net. The focus of

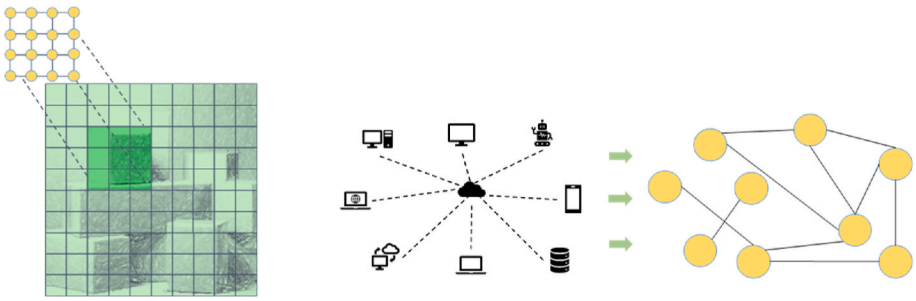


Fig. 1. Two types of data fields.

Section 5 is on the experimental setup and the evaluation of the SAT-Net, and we discuss the practical application in Section 6. Finally, Section 7 concludes the paper with a summary of the findings and discussions.

2. Related work

Handling the encrypted traffic classification problem can be roughly divided into two stages. The first stage is traffic representation, followed by the design of suitable deep learning model classifiers. It can be said that the traffic representation method directly affects the selection of neural network classifiers. We review relevant work based on these two aspects and finally discuss the differences between our work and existing methods. Table 1 provides a summary of the two types of related work.

2.1. Deep learning-based method

With the continuous enhancement of GPU computing power, many fields requiring large-scale training data, such as speech recognition, machine translation, and computer vision, have achieved substantial breakthroughs (Michael et al., 2017). In the field of encrypted traffic classification, Wang et al. (Wei et al., 2017a) first proposed using deep learning models to address the problem of encrypted traffic classification. They utilized the first 784 bytes of packets to learn the nonlinear relationship between input and output through a 1D-CNN (Convolutional Neural Networks). By extracting 10 packets from each flow, each containing 1500 bytes, they obtained the optimal representation of traffic. Yao et al. (Haipeng et al., 2022) achieved VPN traffic type recognition using an LSTM model based on attention mechanisms. Liu et al. (Chang et al., 2019) introduced the Flow Sequence Network (FS-Net), which employs an encoder to map flows to a feature space and then uses a reconstruction mechanism to recover the input sequence. In addition, the method of using a large amount of unlabeled traffic data for pre-training has also achieved good performance as expected. Lin et al. proposed an encrypted traffic classification model ET-BERT (Xinjie et al., 2022) based on the pre-training method, which first performs pre-training on a large-scale unlabeled dataset, and then fine-tunes with a small amount of task-specific dataset. Similarly, using the pre-training method, Lin et al. (Peng et al., 2023) trained a deep learning model based on the self-attention mechanism with the original traffic bytes and packet length sequences, deeply exploring the multimodal relationships between 'bytes-packets-bidirectional flow'. Aceto et al. (Giuseppe et al., 2021) combined pre-training and fine-tuning methods, utilizing multi-modal traffic representation information as input and achieving automatic feature extraction through multi-task learning. Most deep learning-based methods utilize the raw bytes of flows and carefully designed flow or packet features. For instance, Wang et al. proposed App-Net (Xin et al., 2020), an end-to-end hybrid neural network that combines LSTM and CNN to respectively learn the sequence of packet lengths and the raw packets of TLS flows. Some studies have achieved promising results solely using the distribution of packet lengths (Chang et al., 2019) (Chang et al., 2018) (Chen et al., 2020). Their rationale is that the sequence of packet lengths contains temporal characteristics of flows. In general, the stability of traffic features extracted by these methods is susceptible to fluctuations due to differences in network environments, system architectures of applications, and types of services. Additionally, these methods lack focus on the bytes of traffic themselves, hence these features are unstable.

2.2. GNN-based method

In recent years, methods based on graph neural networks, which analyze data from the perspective of non-Euclidean space, have gained widespread attention. Huoh et al. (Ting-Li et al., 2021) first proposed using graph neural networks for traffic classification by considering the relationship between raw bytes of packets from the perspective of non-Euclidean distance space. In subsequent work (Ting-Li et al., 2023), they enhanced graph neural networks by supplementing graph-level attributes. To obtain more accurate traffic graph representations, Jiang et al. (Minghao et al., 2022) proposed using a Flow Relationship Graph to characterize encrypted traffic by defining two types of flow-level relationships. Pang et al. (Bo et al., 2021) characterized network traffic based on packet semantics and causal relationships of network behavior, proposing a Chain Graph as the basis for graph neural network classification. Shen et al. (Meng et al., 2021) found significant differences in the behavior of packets within a single burst and across different bursts, which can serve as the basis for learning features of encrypted traffic by classifiers. Zhang et al. introduced TFE-GNN (Haozhen et al., 2023), a time-fused encoder based on graph neural networks, which learns byte-level traffic graphs of packet headers and payloads through dual embedding and then obtains vector representations of entire packets. Pham et al. (Thai-Dien et al., 2021) learned potential patterns of mobile application traffic using communication

Table 1
Summary of related work on encrypted traffic classification.

Category	Refs.	Year	Model/Algorithm	Traffic features	Goal
Deep Learning-based	Wei et al. (2017a)	2017	1D-CNN	Original traffic byte data in each packet	VPN encrypted traffic
	Chang et al. (2019)	2019	FS-Net	Packet length sequence characteristics	Application Fingerprint
	Xin et al. (2020)	2020	App-Net	Original network traffic flow	Application Fingerprint
	Chen et al. (2020)	2020	Capsule Neural Network	Packet length sequence characteristics	VPN and non-VPN encrypted traffic
	Giuseppe et al. (2021)	2021	DISTILLER	Traffic Object Segmentations	VPN and non-VPN encrypted traffic
	Haipeng et al. (2022)	2022	LSTM & HAN	Original network traffic flow	VPN and non-VPN encrypted traffic
	Xinjie et al. (2022)	2022	ET-BERT	Traffic raw bytes are used for pre-training	Traffic type and Application
	Peng et al. (2023)	2023	PEAN	Traffic raw bytes are used for pre-training	Traffic type and Application
GNN-based	Bo et al. (2021)	2021	CGNN	Chained graph of the chained compositional sequence	Traffic type and Application
	Meng et al. (2021)	2021	GraphDApp	Constructed Traffic Interaction Graph (TIG)	DApp Traffic type and Application
	Thai-Dien et al. (2021)	2021	MAppGraph	A communication graph defined by tuples of IP address and port of the services	Mobile-App Classification
	Minghao et al. (2022)	2022	FG-Net	Constructed Flow Relationship Graph (FRG)	Traffic type and Application
	Ting-Li et al. (2023)	2023	GNNs	Account packet raw bytes, metadata and packet relations	VPN encrypted traffic
	Haozhen et al. (2023)	2023	TFE-GNN	PMI is used to construct byte-level traffic graph	Traffic type and Application
	Tiru et al. (2023)	2023	BehavSniffer	Constructed Traffic Burst Graph (TBG)	Social Network User Behavior
	Ours		SAT-Net	A Packet Byte Graph (PBG) according to the payload and header of the packet	Traffic type and Application

behavior represented as graphs with node features and edge weights, proposing the DGCNN model. It is worth mentioning that in the past two years, research on traffic behavior has also become active in this research community, as identifying traffic behavior is a key research hotspot in the field of encrypted traffic analysis and serves as an important means for network security management and monitoring (Xiaodong et al., 2023). BehavSniffer (Tiru et al., 2023) enhances the relationship between unidirectional flows by using an additional designed burst-bt edge on the basis of GraphDApp (Meng et al., 2021). In (Tiru et al., 2023), the authors focus on the combination of high-order and low-order relationships of flows, and achieve feature fusion using DNN and kernel principal component analysis, ultimately realizing the recognition of user behavior in social media. These methods all construct graphs based on flow or packet features, without distinguishing between communication control behavior and data transmission behavior. Therefore, they exhibit weak robustness against noise in traffic.

In summary, here are the differences between our method and existing works:

a) We observe that a session can be divided into two processes: handshake and teardown phases, and data transmission phase. As shown in Fig. 2, we use a workflow of a TCP session to illustrate the above process. We construct packet byte graphs for the header byte portion (including IP header, TCP header) and payload portion (including data fields) separately, enabling feature extraction at the network packet level. It is worth noting that the applicability of the encrypted traffic classification method proposed in this paper is not limited to the TCP protocol. The reason is that we consider the network topology and the communication process between the client and the server, and regardless of the protocol used, it will include data transmission behavior and communication control behavior (for example, the QUIC (UDP-based) and the UDP (connectionless) also have these two modes).
b) We employ a dynamic sliding window mechanism to construct packet byte graphs, capturing long-term dependencies between bytes and obtaining richer, more accurate representations of traffic graphs.
c) In terms of classifier design, we utilize a multi-head attention mechanism during the feature fusion stage to integrate information about both session control behavior and data transmission behavior. We also validate the generalization of the proposed model on various types of network traffic datasets.

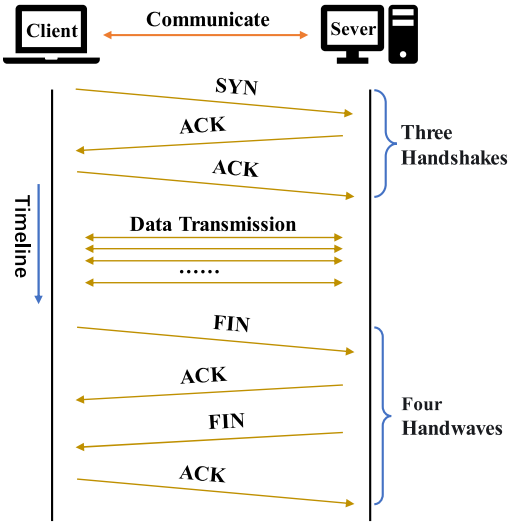


Fig. 2. A TCP session.

3. Preliminary

3.1. Graph based representation of network encrypted traffic

Existing neural networks are well-suited for processing Euclidean data with translational invariance and for uncovering hidden patterns (Bentian et al., 2021). However, a vast amount of data cannot be discussed within the Euclidean space, such as the structure of chemical molecules, user behavior in social networks, and so on. Therefore, researchers have turned their attention to graph-structured data, which are complex and can encapsulate rich information (Ting-Li et al., 2021) (Ting-Li et al., 2023). Below, we will describe the general form of representing traffic data using graph-structured data.

Given a collection of bidirectional flows $S = \left\{f^i\right\}_{i=1}^{n}$, where each flow corresponds to a label $L = \left\{l^i\right\}_{i=1}^{n}$. A packet is the basic unit of a single bidirectional flow. Therefore, each flow f^i in S can be represented as $f^i = \left\{p_j\right\}_{j=1}^{m}$, where each packet p_j consists of several bytes: $p_j = \{b_k\}_{k=1}^{o}$. Encrypted traffic classification aims to categorize an unknown flow or packet into the correct protocol, service type, or application. Therefore, based on the structure of the graph:

$$G = \{V, E, X\}$$

where V represents the set of nodes, E represents the set of edges, $X \in \mathrm{R}^{|V|\times D}$ is the feature matrix of the graph, where D is the dimensionality of node features. $N(n_v) = \{n_v^t\}_{t=1}^{T}$ represents the set of neighbors of node v, and the graph's label is denoted as y. The adjacency matrix of the graph is represented as $A \in \{0,1\}^{|V|\times|V|}$. Where, for any element a_{ij} in A, if $a_{ij} = 1$, it indicates that there is an edge between node i and node j. Thus, A is a symmetric matrix, and all elements on the diagonal are 1.

Clearly, for the task of flow-level encrypted traffic classification, we can treat individual packets or packets within the same burst as nodes, with the relationships between these packets serving as edges (Minghao et al., 2022) (Meng et al., 2021), thereby modeling at the flow granularity. For packet-level classification tasks, we can utilize the bytes of each packet as nodes and the relationships between bytes as edges to capture byte-level features (Haozhen et al., 2023).

3.2. Graph SAmple and aggregate (GraphSAGE)

GraphSAGE (William et al., 2017) does not learn embeddings for individual nodes but learns an aggregation function. This function is used to generate new embedding representations from the local neighborhood of nodes, addressing the inefficiency in training large-scale graph neural networks. Therefore, it is well-suited for handling encrypted traffic. GraphSAGE has three aggregation functions:

(1) Mean Aggregation (MEAN): Similar to the convolution propagation rule in the GCN framework, it directly calculates the mean of neighboring nodes. The calculation is as follows (Equation (1)):

$$p_v^k = \sigma\left(W \bullet mean\left(\left\{p_v^{k-1}\right\} \cup \left\{p_n^{k-1}\right\}\right)\right) \tag{1}$$

where, $\forall n \in N$, p_v^k represents the feature vector of node v at step k, and N represents the set of neighbors.

(2) Long Short-Term Memory Aggregation (LSTM): LSTM aggregation is suitable for scenarios where neighboring nodes are randomly arranged. We demonstrated in experiments that LSTM aggregation yields the best performance for encrypted traffic classification, attributed to its ability to capture temporal information in traffic.

(3) Pooling Aggregation (POOL): The isomorphic max pooling operation of the pooling aggregation achieves aggregation of information across neighbors. The calculation is as follows (Equation (2)):

$$\text{Aggregate}_k^{pool} = max\left(\left\{\sigma\left(W_{pool}p_n^k + b\right), \forall n \in N\right\}\right) \tag{2}$$

As shown in Fig. 3, the general steps of node sampling and aggregation in the GraphSAGE algorithm are as follows: (1) Sample the kth-order neighboring nodes of the center node. (2) Utilize the aggregation function to obtain local neighborhood features. (3) Predict the center node based on the feature information. Due to the ability of the aggregation function to learn new subgraph embedding representations, GraphSAGE is also suitable for learning node representations in graphs with dynamic changes.

4. Methodology

4.1. The design of packet byte graph and construction

Network packets are the most common data units in communication networks and represent the smallest data element in packet-switched networks (PSDN), consisting of control information (such as port numbers, checksums, service types, and protocols) and payload data (Leslie, 2020). Due to the different information carried by the packet header and payload, the former conveys control semantic information about the communication process and the content of the packet, while the latter contains the specific data content of the communication process. Based on this observation, to obtain the global semantics of the packet, we process both the header and payload parts of the packet separately and use the packet byte sequence to construct the Packet Byte Graph (PBG), enabling packet-level encrypted traffic classification. In this graph, nodes represent bytes in the packet, and edges denote the spatial relationships between bytes. The following is a detailed description of our PBG construction process.

4.1.1. Packet byte graph construction

Before constructing the Packet Byte Graph (PBG), we need to preprocess the dataset (the original.pcap files). Firstly, we use the open-source tool, named Splitcap, to extract bidirectional flows based on the packet's five-tuple information. Secondly, we remove packets without payload content. Lastly, we discard the Ethernet header of the packets and anonymize the source and destination IP addresses as well as the port numbers since this information is irrelevant for traffic classification.

Vertex: The vertices of the PBG are the bytes in the packets. Bytes with different values are considered as different vertices, thus limiting the size of the entire graph to within 256 nodes. Each node has two attributes: byte size and count.

Edge: The edges in the PBG are used to represent relationships between vertices. Since the connections between vertices reflect the structural characteristics of the graph data, we adopt the Pointwise Mutual Information (PMI) (Liang et al., 2018) commonly used in NLP tasks to measure the relationships between bytes. To avoid the occurrence of negative infinity in PMI, we calculate the Positive Pointwise Mutual Information (PPMI), which is computed as follows in Equation (3):

$$PPMI(b_1, b_2) = MAX\left(log_2 \frac{P(b_1, b_2)}{P(b_1)P(b_2)}, 0\right) \tag{3}$$

where, $P(b)$ denotes the probability of a byte occurring in all windows. For example, if there are a total of 6 windows and byte b appears in 4 of them, then $P(b) = \frac{2}{3}$. Additionally, $P(b_1, b_2)$ represents the probability of bytes b_1 and b_2 occurring together in a window, that is, out of a total of 6 windows, bytes b_1 and b_2 appear together in 2 of them, thus $P(b_1, b_2) = \frac{1}{3}$. We only construct an edge between two nodes when $PPMI(b_1, b_2) > 0$. It's worth noting that $PPMI(b_1, b_2) = PPMI(b_2, b_1)$, making PBG an undirected graph.

Dynamic Window Setting: Since we handle the header and payload parts of the data packet separately, we dynamically define the window size using a strategy similar to exponential smoothing.

Below, we illustrate the method of setting up a dynamic window using an example. Let's assume a byte sequence {12, f3, 03, 97, 12, 0f, f2, 30}, when the sliding window size is set to $w = 3$ (taking only positive integers), we can calculate that $P(12) = \frac{2}{3}$, $P(97) = \frac{1}{2}$, $P(03) = \frac{1}{2}$, $P(03, 97) = \frac{1}{3}$, $PPMI(97, 03) = log_2 \frac{4}{3}$. We also observe that there may be significant differences between the features of the header and payload, hence a flexible approach is required to set the window size, in order to accommodate different packets. Due to the fixed length of the packet header byte sequence, but not for the payload part, we set different sliding window sizes for these two parts when constructing PBG. Specifically, we set the size of the packet header byte sequence as w_h, and the window size for the payload part is calculated as follows according to equation (4):

$$w_p = \alpha^* MAX\left(\frac{Len_{payload}}{Len_{header}}, w_h\right) + (1 - \alpha)2^{w_h} \tag{4}$$

The dynamic window size can adaptively capture the dependency relationship between bytes in sequences of different lengths. By adjusting α and the window size, we can better adapt to the relationships between bytes in sequences of different lengths. For instance, when the payload length of a packet is much greater than the header length, the window size will be increased accordingly to capture richer byte dependencies. Through a series of experiments, we found that the best performance is achieved when $w_h = 5$ and $\alpha = 0.4$. Algorithm 1

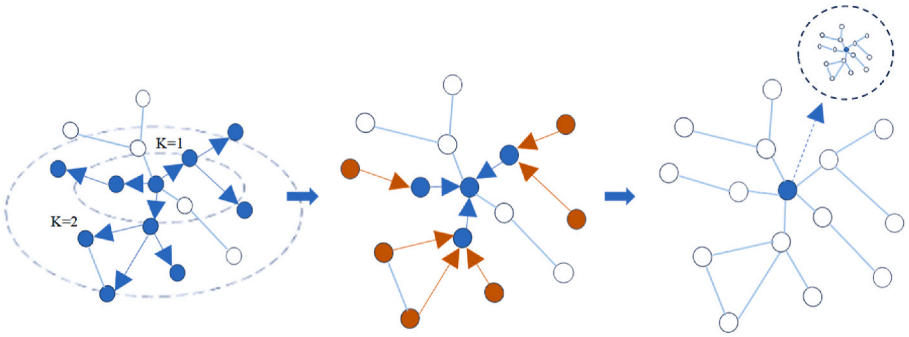


Fig. 3. The GraphSAGE algorithm. Here, the sampled node features are represented as $(y_1, y_2, \ldots, y_m)^T$, which can be used for prediction tasks.

describes the construction process of PBG.

Algorithm 1 Construction of Packet Byte Graph

Input: The byte sequence of the packet: $p = \{b_1, b_2, \ldots, b_n\}$
Output: The Header Byte Graph $G_h = \{V, E\}$ and the Payload Byte Graph $G_p = \{V, E\}$
Initialize $G_h = \{V, E\}$ and $G_p = \{V, E\}$ as empty sets
Separate the sequence p into header and payload
for $b_i \in p$ **do**
  if $i \leq length(header)$ **then**
    Add vertices in V_{G_h} according to ascending order of byte value
    if $\forall v_r, v_s \in V_{G_h}$ **then**
      if $PPMI(v_r, v_s) > 0$ **then**
        Add an edge between v_r, v_s in E_{G_h}
  else
    Add vertices in V_{G_p} according to ascending order of byte value
    if $\forall v_k, v_l \in V_{G_p}$ **then**
      if $PPMI(v_k, v_l) > 0$ **then**
        Add an edge between v_k, v_l in E_{G_p}
return G_h and G_p

4.1.2. A PBG sample of SAT-Net

In this subsection, we visualize the construction process of a PBG through an example. As depicted in Fig. 4, the process begins by calculating the frequency of each byte (same value) within all sliding windows of the byte sequence (steps 1-n). Subsequently, bytes are mapped to vertices in the graph, and relationships between vertices are determined by computing Positive Pointwise Mutual Information (PPMI). This results in the formation of the PBG. For convenience, we arrange the byte values in sequence; however, such a logical order may not exist in the actual graph. Then, we differentiate the graph into PBG-header and PBG-payload based on the header and payload in the byte sequence.

4.2. SAT-Net

Prior to this section, we have addressed the conversion of network traffic data into graph data, transforming packet raw byte sequences into undirected Packet Byte Graphs (PBGs), where each byte in a packet constitutes a vertex in the graph, and the connections between vertices represent the associations between bytes. Additionally, we have utilized a window-based optimization method to obtain a more accurate representation of the packet byte graph. Following this, we present the specific design concept of SAT-Net, Fig. 5 provides an overall framework of SAT-Net.

Specifically, SAT-Net comprises four modules: a graph embedding layer, feature remapping layer, Staggered Attention layer, and classification layer, which are interconnected to form an end-to-end network. Initially, we employ GraphSAGE to embed PBG-header and PBG-payload in parallel, obtaining the vector representation of PBG in the feature space. Then, each embedding is remapped through an MLP to obtain more semantic information about the PBG. Subsequently, staggered attention layer fuses the vector representations of PBG-header and

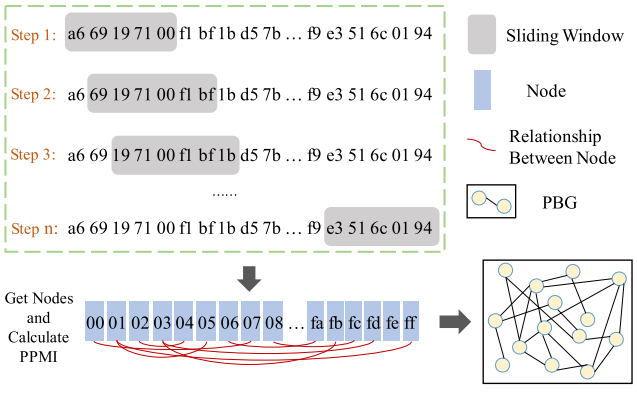


Fig. 4. Visualization of the construction process of the PBG.

PBG-payload using a multi-head attention mechanism to capture important features of the entire packet. Finally, the fused representation is input into a classification layer to obtain the final label.

4.2.1. Graph embedding layer

Having constructed the PBG, we have obtained a graph-structured representation of the encrypted traffic, where nodes represent bytes in packets and edges denote the relationships between bytes. The core objective of the graph embedding layer is to encode the Packet Byte Graph (PBG) into high-dimensional feature vectors to capture the complex relationships and patterns between packet bytes. In SAT-Net, we utilize the GraphSAGE algorithm as the basis for graph convolution operations, owing to its unique sampling and aggregation strategy that efficiently handles large-scale graph data, particularly suitable for complex graph structures in encrypted traffic analysis. The GraphSAGE algorithm is an inductive learning method, offering several significant advantages over traditional full-graph convolution methods. Firstly, GraphSAGE effectively limits the neighborhood size required for each computation through its fixed-interval node sampling strategy, thereby accelerating feature propagation and significantly improving computational efficiency. This is especially critical in scenarios such as encrypted traffic analysis, where the data volume is large and highly diverse. Secondly, GraphSAGE's aggregation operation effectively captures hidden features between nodes by representing the features of a node and its neighbors in a joint embedding space. Particularly in our encrypted traffic analysis task, GraphSAGE iteratively aggregates the feature representations of nodes and their neighbors, integrating local features of the traffic byte graph (PBG) into global features. This iterative aggregation process starts with the sampled nodes and their neighbors, calculating the feature vectors of these nodes and aggregating them into a specific neighborhood. Then, the current representation of the node is concatenated with the aggregated neighborhood vector and input into a nonlinear activation function (such as PReLU), generating the final embedding vector of the node. This embedding vector better captures both local and global features within the packet, thereby providing strong support for subsequent feature remapping and classification.

The input of the graph embedding layer is defined as $G = \{V, E\}$, where the number of nodes in the graph is n, V represents the set of nodes, and E is the set of edges. Node features are denoted as $\{x_v, \forall v \in V\}$. To further learn the deep relationships between graph nodes, we stack two GraphSAGE convolutional layers in the graph embedding layer. The output of the first layer is passed as the hidden state $\widetilde{H}_t^1$ to the next layer. Finally, the output layer uses the $\widetilde{H}_t^2$ from the second layer for batch normalization and regularization to compute the final feature vector v. The computation process is as follows in Equations (5) and (6):

$$\widetilde{H}_t^{(k)} = SAGEConv\left(H_t^{(k)}\right), 1 < t < n \tag{5}$$

$$v = \sigma\left(W_o \cdot CONCAT\left(\widetilde{H}_t^{(1)}, \widetilde{H}_t^{(2)}\right)\right) \tag{6}$$

where σ is the activation function, W_o represents the weight matrix for a linear transformation, which can be used to change the output dimension. We use average pooling to obtain the feature vector of PBG. It is worth noting that our SAT-Net adopts a parallel embedding approach, where the parameters of the model are independent for PBG-header and PBG-payload during the embedding learning process.

4.2.2. Feature remapping layer

Inspired by the graph isomorphism problem (Jin-yi et al., 1992), we designed an MLP in this layer to learn the representation vector of the PBG for feature enhancement. It comprises two linear layers, a batch normalization layer, and a parameterized PReLU activation function, with each linear layer acting as a recursive neighbor aggregation strategy, performing deep processing and feature enhancement on node

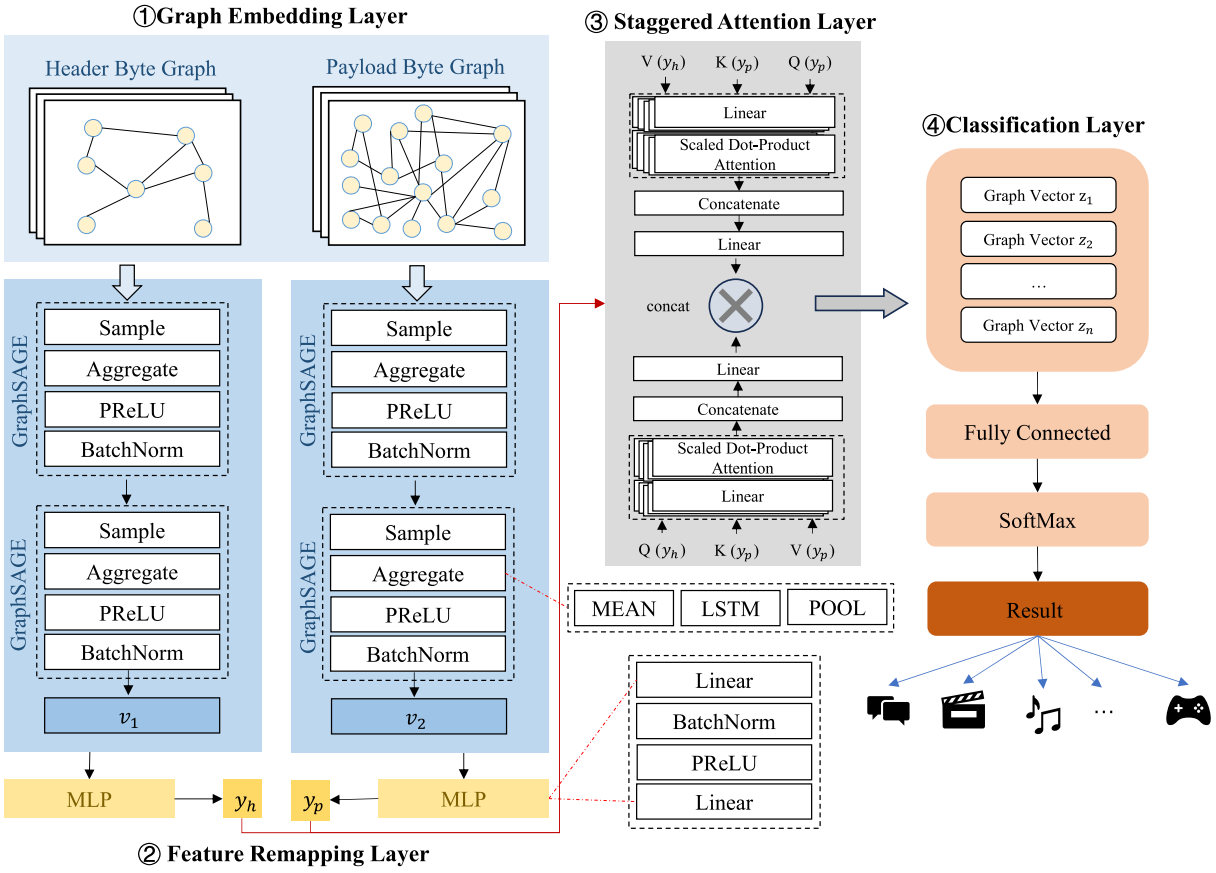


Fig. 5. Overall network architecture of the proposed SAT-Net.

features. Here is our design approach for the MLP:

Within the MLP, the input data first undergoes a linear transformation through the first linear layer, mapping node features to a hidden feature space. This step expands the original feature information to a higher-dimensional representation through linear transformation, thereby preserving and enhancing the initial information of the node and its neighborhood. After the linear transformation, the batch normalization layer normalizes the features by adjusting their mean and variance, ensuring that the hidden layer features have zero mean and unit variance. This operation not only accelerates the training process of the network but also significantly improves the stability and convergence speed of the model, avoiding issues such as vanishing or exploding gradients, thus maintaining the consistency and robustness of the feature representation. Subsequently, the batch normalization layer normalizes the output of the linear layer. Normalization aids in accelerating the training process and improving the stability and convergence speed of the model. Unlike traditional ReLU, PReLU enhances the network's ability to handle nonlinear problems by adaptively adjusting the output of negative input values. This is particularly effective when the distribution of input data is uneven, as PReLU can better capture potential feature relationships. After passing through the second linear layer and the activation function, the output layer of the MLP maps these nonlinearly transformed features to the final output space. This recursive neighbor aggregation strategy ensures that the model can effectively integrate multi-level information from nodes and their neighborhoods, strengthening the model's ability to recognize key features. Formally, the computational formula for the above process is given by (7)-(12):

(1) Linear Transformation:

$$h_1 = W_1 h_v + b_1 \tag{7}$$

where, h_v is the input node feature vector, $W_1 \in \mathrm{R}^{m\times n}$ and $b_1 \in \mathrm{R}^m$ are the weight matrix and bias vector of the linear layer, respectively, and h_1 is the output of the linear layer.

(2) For each feature dimension i of h_1, compute the mean μ_i and variance σ_i^2.

$$\mu_i = \frac{1}{B}\sum_{k=1}^{B} h_{1,k,i} \tag{8}$$

$$\sigma_i^2 = \frac{1}{B}\sum_{k=1}^{B} \left(h_{1,k,i} - \mu_i\right)^2 \tag{9}$$

where B represents the batch size. Then, after normalization, we obtain h_2:

$$h_2 = \frac{h_1 - \mu}{\sqrt{\sigma^2 + \epsilon}}\cdot\gamma + \beta \tag{10}$$

where $\gamma \in \mathrm{R}^m$ and $\beta \in \mathrm{R}^m$ are learned scaling and shifting parameters, respectively. The purpose of ϵ is to avoid division by zero.

(3) PReLU activation function

$$h_3 = max(0, h_2) + a^{*}min(0, h_2) \tag{11}$$

(4) output:

$$y = W_2 \cdot h_3 + b_2 \tag{12}$$

Transform h_3 through the weight matrix W_2 and bias b_2 for the second linear transformation to obtain the final output y.

Through the above steps, the Feature Remapping Layer enhances the

feature representation while maintaining the integrity of the original information. It is able to capture global and local features in the PBG more effectively, ultimately resulting in the remapped feature vector y. Since we perform parallel embeddings, we distinguish between the feature vectors of PBG-header and PBG-payload, for subsequent descriptions, denoted as y_h and y_p respectively.

4.2.3. Staggered attention layer

Now that we have obtained the rich vector representation of the PBG, we use the Staggered Attention Layer to capture traffic feature information in multiple semantic spaces through the multi-head attention mechanism. The aim is to address the issue of information loss in traditional feature concatenation methods and to enhance the model's ability to express complex features. The benefits of this approach are as follows:

(1) Context-aware Feature Fusion: The core advantage of the attention mechanism lies in its ability to provide contextual information for each feature vector, allowing the fusion of features to consider the global dependencies within the Packet Byte Graph (PBG). Through a weighted approach, the attention mechanism automatically assigns higher weights to important features while diminishing the impact of irrelevant or redundant features.
(2) Multi-perspective Feature Representation with Multi-head Attention: The multi-head attention mechanism processes multiple attention heads in parallel, enabling the model to learn feature representations in multiple subspaces. Each attention head focuses on different types of feature relationships within a distinct representation subspace, capturing the diversity and complexity of traffic features. For example, one attention head may concentrate on specific patterns in header information, while another may focus on detailed features in the payload. This multi-perspective feature representation allows the model to more comprehensively capture rich information in encrypted traffic, enhancing classification accuracy and robustness.
(3) Avoiding Information Loss and Refined Feature Fusion: Directly concatenating header and payload feature vectors can lead to information loss due to dimensional discrepancies or uneven weight distribution. Our Staggered Attention Layer refines the weighted fusion of these features through the multi-head attention mechanism, ensuring a more rational weight allocation between different features in the model. This weighted fusion not only effectively retains the original information of each feature vector but also further strengthens the inter-feature correlations.

Assuming the number of attention heads is h, the output of the Head_i can be represented as equation (13):

$$\text{AttnHead}_i(\text{Q},\text{K},\text{V}) = \text{softmax}\left(\frac{\text{Q}\cdot K^T}{\sqrt{d_k}}\right)\cdot \text{V} \tag{13}$$

where d_k is the dimensionality of the Key vectors. The output of the multi-head attention mechanism is represented as equation (14):

$$MultiHead_{Attn(Q,K,V)} = Concat(AttnHead_1, \ldots, AttnHead_h)\cdot W^O \tag{14}$$

where W^O is the weight matrix of the output layer.

Then, we compute two attention outputs separately: one using the PBG-header feature vectors y_h as Query and the PBG-payload feature vectors y_p as Key and Value; the other using the PBG-payload feature vectors y_p as Query and the PBG-header feature vectors y_h as Key and Value. Then, we concatenate these two outputs as the final output z, and the computation process is as follows in equations (15)–(17):

$$\widetilde{y}_h = MultiHead\_Attn\left(y_h, y_p, y_p\right) \tag{15}$$

$$\widetilde{y}_p = MultiHean\_Attn\left(y_p, y_h, y_h\right) \tag{16}$$

$$\text{z} = \text{CONCAT}\left(\widetilde{y}_h, \widetilde{y}_p\right) \tag{17}$$

In conclusion, the Staggered Attention Layer effectively enhances the model's feature representation capability through the context-awareness of the multi-head attention mechanism, multi-perspective feature representation, refined feature fusion, and information capture across semantic spaces. This ensures that key features can be accurately captured and integrated in complex encrypted traffic classification tasks.

4.2.4. Classification layer

The classification layer consists of two fully connected layers. The first fully connected layer accepts the output z from the staggered attention layer as input and employs the PReLU activation function, which adaptively adjusts the shape of the activation function to enhance the model's non-linear expressive capabilities. The second fully connected layer is a linear layer that takes the output of the first fully connected layer as input, and its output feature dimension corresponds to the number of categories in the classification task. This layer serves to map the output features of the previous layer to the final classification results.

The output o of the entire classification layer is the output of the second fully connected layer, representing the model's predictions for the input data classification. In this way, the model can make classification decisions based on the input data features.

Loss function. Considering that traffic data in real-world traffic environments is often imbalanced, the use of standard cross-entropy loss, commonly employed in classification problems, may lead to biased estimates. Some research has utilized Generative Adversarial Networks (GAN) (Wajdi and Wonjun, 2020) to generate traffic samples to address the issue of sample data imbalance. However, these generated samples often do not align with real samples and can increase the training cost. Therefore, we employ Focal Loss as the loss function. Formally, it is represented by Equation (18):

$$FL_{softmax}(g_t) = -\alpha_t(1-g_t)^{\gamma}\log\left(g_t\right)\big) \tag{18}$$

where g_t represents the SoftMax prediction for class α_t is the weight for class t, and γ is the tuning factor.

In this paper, we sum up the losses for all classes, then average them to obtain the final loss value. The calculation process is as follows:

$$Loss = -\sum_{t-1}^{C}\alpha_t\left(1-g_{i,t}\right)^{\gamma}log\left(g_{i,t}\right)\Big) \tag{19}$$

where, C represents the number of classes, $g_{i,t}$ denotes the probability predicted by the *softmax* function for sample i to belong to class t.

5. Experiment and evaluation

In this section, we evaluate the effectiveness of SAT-Net on the task of encrypted traffic classification through a series of experiments. Initially, we introduce the datasets and comparison baselines used, followed by an overall performance assessment via comparative experiments with state-of-the-art methods (Section 5.2 **Comparison Experiments**). Subsequently, we analyze the necessity of each component within SAT-Net (Section 5.3 **Ablation Study**). We then conduct a sensitivity analysis by varying different hyperparameters of the model (Section 5.4 **Sensitivity Analysis**), and finally, we study the balance between the model's training and testing costs and its performance (Section 5.5 **Tradeoff Study for Cost and Performance**).

5.1. Experiment setup

5.1.1. Datasets

The following paragraphs introduce the four public datasets and one self-collected dataset used to evaluate the performance of SAT-Net.

(1) ISCX VPN-nonVPN (Gerard et al., 2016): This dataset, provided by the Information Security Center of Excellence (ISCX), is specifically designed for research on the classification of VPN (Virtual Private Network) and non-VPN traffic. It encompasses a variety of traffic types and applications, (such as Chat: Skype, ICQ; VoIP: Facebook, Hangout), among others. The ISCX VPN-nonVPN dataset is generated by simulating the activities of users Alice and Bob in different network environments. Although it covers a variety of traffic types and applications, these simulated environments may differ from real-world network conditions, potentially leading to data that is not comprehensive or representative in certain aspects.
(2) ISCX Tor-nonTor (Arash Habibi et al., 2017): Similar to ISCX VPN-nonVPN, this dataset, also provided by ISCX, is dedicated to the classification of Tor (The Onion Router) and non-Tor traffic. Due to the unique traffic characteristics of the Tor network, which are often employed for privacy and anonymous communication, this dataset is particularly relevant for studies that aim to differentiate between legal and illicit activities. However, in this dataset, encrypted traffic predominantly uses the Tor protocol, which somewhat limits the dataset's applicability and effectiveness in handling other encryption protocols such as VPN and IPSec.
(3) USTC-TFC2016 (Wei et al., 2017b): Developed by USTC (University of Science and Technology of China), this dataset includes both malicious and benign traffic for research on traffic classification. It comprises network traffic from a multitude of applications, ranging from web browsing and social media to email and various online services. Although the USTC-TFC2016 dataset contains network traffic data from a variety of applications, its scale and diversity may still not be sufficient to cover all types of traffic found in actual network environments, which could potentially limit the model's generalization ability in practical applications.
(4) CIC IoT (2023) (Euclides Carlos Pinto et al., 2023): This dataset, provided by the Canadian Institute for Cybersecurity (CIC), focuses on research related to Internet of Things (IoT) devices and associated security threats. It includes network traffic collected from various IoT devices and services, encompassing both normal operations and potential attack traffic, such as DDoS attacks and malware distribution. However, the variety, quantity, and configurations of devices in actual IoT environments may be more complex, which could potentially impact the model's generalization capability in practical applications.
(5) HTTPS-D: We collected 14 web traffic deployments with TLS 1.3 from different types and services of applications using open-source tools Wireshark and tcpdump on a dedicated server. The dataset encompasses services such as news websites, social media, video platforms, and more. Due to the deployment of collection devices at the campus network gateway, with the server location relatively centralized, it may not reflect the user experience and network conditions in different geographical locations.

Dataset Preprocessing. Before using the aforementioned datasets, we need to perform a series of preprocessing operations to meet the requirements for training in SAT-Net. First, we use Splitcap to segment the raw PCAP files for each category of traffic and delete empty flows and short flows. Next, we utilize the Scapy library in Python to anonymize the packets within the flows, preventing the model from being biased by specific information. It is worth noting that although we strive to eliminate useless traffic flows, some irrelevant traffic may still remain; however, this noise does not significantly affect the training and evaluation of our model. Notably, since we are conducting packet-level classification tasks, we select the first 10 packets of each traffic flow as the input for the model. This amount of information has been proven to be sufficient in numerous studies on encrypted traffic classification (Wei et al., 2017a) (Peng et al., 2023). Table 2 provides statistical information for these datasets after preprocessing.

5.1.2. Baselines

We selected six advanced models as comparison baselines, which can be categorized into two groups: deep learning-based methods (FS-Net (Chang et al., 2019), DeepPacket (Mohammad et al., 2019), FlowPic (Tal and Yuval, 2021)), and GNN-based methods (GraphDApp (Meng et al., 2021), FG-Net (Minghao et al., 2022), EC-GCN (Zulong et al., 2023)). It is worth noting that these methods can also be divided into three different traffic representation approaches: based on raw byte sequences (DeepPacket, FS-Net), converting encrypted traffic into images (FlowPic), and transforming encrypted traffic into graph classification problems (GraphDApp, FG-Net, EC-GCN), all of which are exemplary representatives among these methods. Below is a brief introduction to these models:

(1) FS-Net: This method learns features from raw flows and employs a multi-layer encoder-decoder structure for feature enhancement through reconstruction.
(2) DeepPacket: Utilizing autoencoders and convolutional neural networks for processing network traffic, this approach can simultaneously differentiate between application types and service types.
(3) FlowPic: This method leverages temporal and size features of flows, converting traffic data into images without utilizing payload data to form a traffic images.
(4) GraphDApp: Building a Traffic Interaction Graph (TIG) and combining it with a GNN-based classifier specifically designed for the identification of Decentralized Applications.
(5) FG-Net: This model uses information-enhanced graph structures to design a GNN-based traffic fingerprint learning model.
(6) EC-GCN: A temporal-spatial (multi-subgraph) classification architecture that utilizes graphical layers and graph learning layers to extract features from encrypted traffic.

5.1.3. Metrics

To evaluate the performance of SAT-Net, we utilize four metrics commonly employed in classification problems: Accuracy, Precision, Recall, and the F1 Score. Accuracy serves as an overall measure of the model's performance. Precision and Recall are utilized to assess the model's efficiency for each traffic category. The F1 Score, calculated as the harmonic mean of Precision and Recall, provides an integrated measure of the model's performance across all categories. The calculation methods for these metrics are presented in Equations (20)–(23):

Table 2
Statistics of experimental datasets. *In the USTC-TFC2016, both malware and benign have 10 categories each; due to the vast size of the entire CIC IoT (2023) dataset, we selected a portion of representative encrypted traffic from attacks for the experiment, including DDoS-ICMP_Flood, DoS-SYN_Flood, Recon, Mirai-Arpspoofing, MITM, VulnerabilityScan, and SqlInjection.

Dataset	Number of Types	Total number of biflows	Dataset size
ISCX VPN-nonVPN	14	79315	16.3 GB
ISCX Tor-nonTor	7	50613	14.2 GB
USTC-TFC2016	10/10	172048	3.2 GB
CIC IoT (2023)	7	132971	20.3 GB
HTTPS-D	14	11594	4.3 GB

$$ACC = \frac{TP + TN}{TP + FP + FN + TN} \tag{20}$$

$$PR = \frac{TP}{TP + FP} \tag{21}$$

$$RC = \frac{TP}{TP + FN} \tag{22}$$

$$F1 = 2^{*}\frac{precision^{*}recall}{precision + recall} \tag{23}$$

ACC refers to the proportion of correctly classified samples out of the total number of samples. TP and FN represent true positives and true negatives, respectively, while FP and FN denote false positives and false negatives, respectively. A higher PR value indicates that the model has a higher capability to discriminate traffic of that category, while a larger RC value suggests that the model has a lower false positive rate for traffic of that category. The F1 score is defined as the arithmetic mean of PR and RC, and we utilize the F1 score to evaluate the model's overall balanced classification performance.

5.1.4. Implementations

Our experimental environment consists of an Ubuntu 18.04 operating system, an Intel® Xeon® Platinum 8255C CPU @2.50 GHz with 43 GB of memory, and an RTX 3090 (24 GB) GPU. The model construction was completed using the Python 3.8 within the PyTorch1.10 environment. The processed dataset was randomly divided into training, validation, and testing sets in a ratio of 8:1:1. Additionally, 10-fold cross-validation was employed during training to reduce randomness and mitigate the limitations associated with fixed dataset splits, thereby enhancing the model's generalization capabilities. During the training process, the AdamW optimizer was utilized to optimize all parameters, with the learning rate set to 1e-4, batch size fixed at 32, and dropout rate at 0.1. SAT-Net introduces early stopping mechanisms and a learning rate scheduler to dynamically adjust the learning rate during training, preventing overfitting and enhancing the model's convergence speed and stability. Additionally, all experimental settings have undergone strict consistency checks to ensure that the same configuration is maintained across all cross-validation folds, eliminating potential experimental biases and inconsistencies, thereby increasing the reliability and reproducibility of the results.

5.2. Overall performance evaluation via comparison experiment

To validate the performance of our proposed SAT-Net, we compared it with six baseline models on five datasets, with all baseline models reproduced according to the original content's network structure and parameter settings. Our SAT-Net achieved the best performance on all datasets, and the following are the model's performances in each dataset:

(1) ISCX VPN-nonVPN. Table 3 shows that SAT-Net outperforms the second-best model (DeepPacket) by 8.72% in accuracy and by 10.31% in F1-score on the ISCX VPN-nonVPN dataset for VPN traffic identification. Therefore, the packet-level classification performed by SAT-Net and DeepPacket helps mitigate the issue of dataset imbalance. Moreover, FG-Net achieves the second-best performance on non-VPN traffic, indicating the superiority of GNN-based methods.

(2) ISCX Tor-nonTor. Due to the use of onion routing technology, Tor traffic is encrypted layer by layer, making it more difficult to identify. As shown in Table 4, SAT-Net still maintains a high accuracy rate. For Tor traffic identification, it outperforms the second-best FG-Net by 4.81% in accuracy and by 2.03% in F1-score, indicating that the use of graph-structured data can accurately represent Tor encrypted traffic. Notably, the FS-Net, with its complex reconstruction mechanism, performs almost identically to FG-Net in Tor traffic identification.

(3) USTC-TFC2016. This dataset contains both malicious software traffic and benign traffic. From Table 5, it can be seen that the accuracy of all models is less than 90%, with SAT-Net's accuracy being only 89.03%, indicating that the behavior patterns of malicious software traffic are similar, making identification difficult. Notably, GraphDApp performs poorly on this dataset, suggesting that the interaction patterns of decentralized Applications are quite different from those of malware.

(4) CIC IoT (2023). As shown in Table 6, similar to Tor traffic, the performance of all models on this dataset is not satisfactory. Our SAT-Net achieved the best result, with an accuracy of only 83.41%, followed by FlowPic at 81.38%. We observed that network traffic attacks in IoT have fixed patterns, so the approach of modeling traffic as images in FlowPic to construct traffic fingerprints can also yield good results.

(5) HTTPS-D. This dataset is characterized by a large number of samples and a diverse range of traffic types. As shown in Table 6, our model achieved the top performance with an accuracy of 99.59% and an F1 score of 99.79%. Methods based on GNN all achieved accuracies above 90%, with the exception of GraphDApp. Due to its integration of both communication control behavior and data transmission behavior from bidirectional flows, SAT-Net exhibits strong resistance to interference. Consequently, when the dataset is sufficiently large, SAT-Net can effectively mine the implicit patterns within encrypted traffic using PBG without being swayed by irrelevant traffic. This suggests that SAT-Net is capable of adapting to concept drift.

5.3. Ablation study

In this section, we validate the influence of each component within SAT-Net by conducting ablation studies on the HTTPS-D dataset. The results of these experiments are presented in Fig. 6. SAT-L2 denotes the model without the feature remapping layer (L2), while SAT-L3 signifies the model without the multi-head attention layer (L3). Additionally, we substituted SAGEConv with either a Graph Convolutional Network (GCN) to produce SAT-GCN or a Graph Attention Network (GAT) to produce SAT-GAT, thereby assessing the efficacy of SAGEConv within SAT-Net. From these ablation studies, we can draw the following

Table 3
Experimental results on ISCX VPN-nonVPN.

Method	ISCX-VPN				ISCX-nonVPN			
	ACC	PR	RC	F1-score	ACC	PR	RC	F1-score
FS-Net	0.8621	0.9773	0.8607	0.9149	0.7079	0.9196	0.7465	0.7586
DeepPacket	0.8713	0.855	0.8748	0.8647	0.7701	0.7742	0.9603	0.8571
FlowPic	0.8538	0.9363	0.7917	0.8566	0.6944	0.7073	0.7310	0.7131
GraphDApp	0.4212	0.4531	0.4852	0.4641	0.4087	0.4119	0.5971	0.5897
FG-Net	0.8592	0.8313	0.8519	0.8466	0.8077	0.8519	0.9545	0.8753
EC-GCN	0.7338	0.7214	0.7401	0.7291	0.6572	0.7509	0.7121	0.7211
SAT-Net	0.9605	0.9616	0.9725	0.9678	0.9500	0.9523	0.9408	0.9494

Table 4
Experimental results on ISCX Tor-nonTor.

Method	ISCX-Tor				ISCX-nonTor			
	ACC	PR	RC	F1-score	ACC	PR	RC	F1-score
FS-Net	0.8911	0.9029	0.9042	0.9031	0.9153	0.9153	0.9141	0.9104
DeepPacket	0.7071	0.7417	0.6209	0.7313	0.8741	0.8653	0.7792	0.8167
FlowPic	0.7199	0.7311	0.7801	0.7484	0.5243	0.5593	0.6074	0.5653
GraphDApp	0.6686	0.5349	0.4799	0.5097	0,8605	0.7743	0.7193	0.7627
FG-Net	0.8914	0.9253	0.9615	0.9434	0.7895	0.6714	0.6615	0.6613
EC-GCN	0.4971	0.4915	0,4376	0.4811	0.4103	0.4253	0,3752	0.3951
SAT-Net	0.9532	0.9599	0.9652	0.9637	0.9272	0.9182	0.9746	0.9621

Table 5
Experimental results on USTC-TFC2016.

Method	USTC-TFC2016-malware				USTC-TFC2016-benign			
	ACC	PR	RC	F1-score	ACC	PR	RC	F1-score
FS-Net	0.6889	0.6679	0.6815	0.6722	0.6576	0.6594	0.6465	0.6486
DeepPacket	0.8713	0.8851	0.8248	0.8747	0.7451	0.7178	0.7354	0.7187
FlowPic	0.5538	0.5951	0.5917	0.5567	0.5942	0.54973	0.5313	0.5136
GraphDApp	0.1231	0.1341	0.1456	0.1403	0.1731	0.1354	0.1597	0.1373
FG-Net	0.7129	0.8077	0.8109	0.8027	0.6667	0.6532	0.6664	0.6567
EC-GCN	0.8863	0.8824	0.8404	0.8591	0.8347	0.8591	0.8112	0.8225
SAT-Net	0.8903	0.9403	0.9152	0.9201	0.9323	0.9597	0.9147	0.9234

Table 6
Experimental results on CIC IoT (2023) and HTTPS-D.

Method	CIC IoT (2023)				HTTPS-D			
	ACC	PR	RC	F1-score	ACC	PR	RC	F1-score
FS-Net	0.4289	0.8179	0.8215	0.8122	0.8576	0.8594	0.8365	0.8483
DeepPacket	0.5692	0.5052	0.5513	0.5502	0.7551	0.7478	0.7354	0.7387
FlowPic	0.8138	0.7843	0.8228	0.7916	0.8847	0.9346	0.9036	0.9131
GraphDApp	0.1331	0.1901	0.1582	0.1599	0.7901	0.7882	0.7438	0.7773
FG-Net	0.7129	0.7003	0.7409	0.7123	0.9592	0.9637	0.9435	0.9551
EC-GCN	0.7363	0.7224	0.7504	0.7231	0.9394	0.9725	0.9641	0.9652
SAT-Net	0.8341	0.8213	0.8016	0.8141	0.9959	0.9909	0.9921	0.9979

Table 7
Experimental results of ablation study on HTTPS-D.

Method	HTTPS-D			
	ACC	PR	ACC	F1-score
SAT-L2	0.7331	0.7421	0.7318	0.7378
SAT-L3	0.5328	0.5704	0.5317	0.5591
SAT-GCN	0.7459	0.7502	0.7417	0.7487
SAT-GAT	0.8227	0.8617	0.8383	0.8568
SAT-Net	0.9959	0.9909	0.9921	0.9979

conclusions:

(1) We initiated by examining the performance fluctuations induced by the alteration of L2 and L3. Table 7 illustrates that the removal of either L2 or L3 results in substantial decrements in model performance, as measured by ACC and F1-score. Specifically, ACC decreases by 26.28% and 46.31%, respectively, and the F1-score decreases by 26.01% and 43.88% respectively. This evidence suggests that both L2 and L3 contribute significantly to the model's performance. L2, implemented as a multi-layer perceptron (MLP), functions to project the PBG into different vector spaces, analogous to feature propagation. When the PBG's representation capacity suffices, the model can, in the absence of L2, utilize the multi-head attention mechanism within L3 to fuse features and elicit rich semantic representations. In particular, the model's optimal performance is contingent upon the fusion of information from PBG-header and PBG-payload, underscoring the pivotal role of L3 over L2.

(2) Analysis of Table 7 reveals that when the model employs GCN or GAT for convolutional operations, there is a significant drop in performance, as indicated by ACC and F1-score. Specifically, ACC decreases by 25% and 17.32%, respectively, and the F1-score decreases by 24.92% and 14.11%, respectively. This decline is attributed to the fact that GCN does not utilize an adjacency matrix that includes neighbor nodes, whereas the construction process of PBG emphasizes the role of node neighbors. Interestingly, there is also a performance degradation when SAGEConv is replaced with GAT. GAT relies on the weights of neighbor nodes to update the information of the central node, and we have also employed attention mechanisms at L3. This suggests that SAT-GAT experiences overfitting.

5.4. Sensitivity analysis

In this section, we selected several hyperparameters to study their impact on model performance. The choice of these hyperparameters for analysis is based on their significant influence on the core mechanisms and performance of the model, while other hyperparameters are relatively less important or contribute less to the model's performance. The experiments were conducted on the HTTPS-D dataset and the USTC-TFC2016(benign) dataset with the largest number of sample flows. We chose ACC as the evaluation metric, where ACC-1 represents the experimental results on HTTPS-D, and ACC-2 represents the results under USTC-TFC2016-benign.

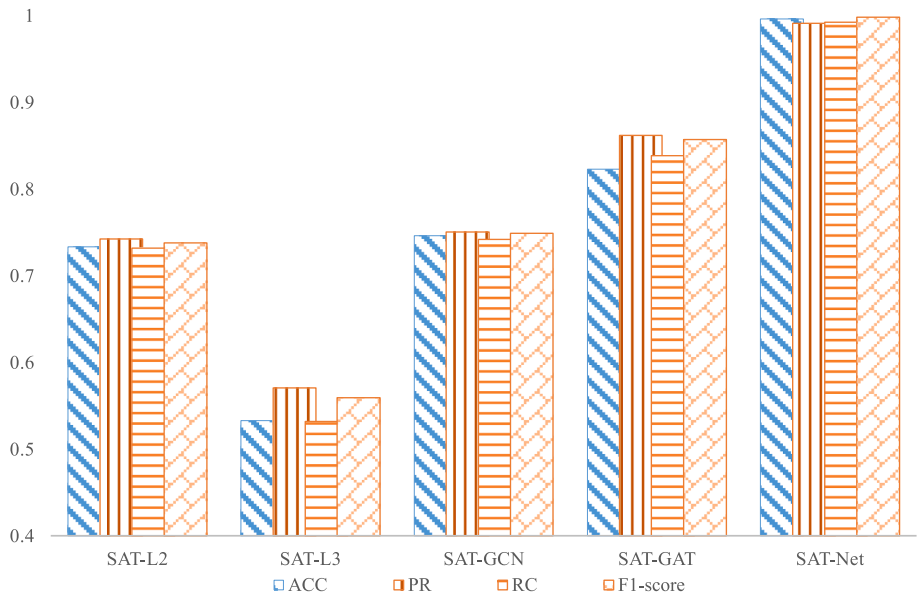


Fig. 6. Results of ablation experiments on the HTTPS-D dataset.

(1) Sliding Window Size Index (α): This hyperparameter primarily affects the structure of the PBG. As α approaches 0, the PBG tends to be constructed from sliding windows of a fixed size; as α approaches 1, the sliding window size is more influenced by the length of the packet payload. The results shown in Fig. 7(a) indicate that, since the HTTPS-D dataset consists of web traffic, the packet length distribution across different website services possesses learnable characteristics (Ziqing et al., 2019). This suggests that the window size should be more closely related to the correlation of the packet length distribution. Similarly, such a pattern is also observed in USTC-TFC2016-benign, indicating that the PBGs constructed by us are more inclined to describe the recognizable patterns of traffic bytes from the distribution regularities of packet lengths.

(2) Learning rate: A learning rate that is too large can make it difficult for the model to converge, while too small a learning rate can increase the convergence time of the model. Fig. 7(b) represents the relationship between the learning rate and the model's accuracy. For the HTTPS-D dataset, the epoch size is set to 21; for the USTC-TFC2016-benign dataset, the epoch size is set to 14. The results show that a learning rate of 1e-4 achieves the highest accuracy, and a learning rate lower than this cannot achieve the expected effect within the same time.

(3) SAGEConv Layer Numbers: SAT-Net performs graph convolution operations by stacking multiple SAGEConv layers. Fig. 7(c) illustrates the impact of different numbers of SAGEConv layers on the model's accuracy. It is evident that when only one layer is used, the model lacks learning capacity and exhibits a lower accuracy. When the number of layers exceeds two, the model's accuracy begins to decline, which is indicative of overfitting.

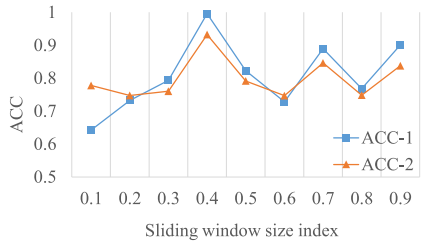
(4) Head Numbers of the Multi-Head Attention: The number of heads in the multi-head attention mechanism determines how many semantic spaces the model employs to learn the features of encrypted traffic. As shown in Fig. 7(d), when the number of heads is too few, the model's accuracy decreases by more than 12%. The highest accuracy is achieved when the number of heads is 8. When the head count is increased to 16 or 32, the accuracy drops, indicating that 8 heads are sufficient to fully leverage the expressive power of PBG.

5.5. Tradeoff study for cost and performance

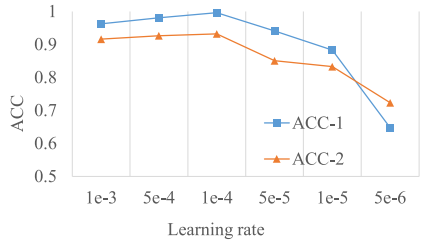
In practical applications, striking a balance between training costs and model performance is of paramount importance. Overzealous pursuit of higher accuracy often leads to resource wastage. Therefore, in this section, we analyze the trade-off between model performance and training costs. The experiments were conducted on the HTTPS-D dataset and USTC-TFC2016(benign), and the following is a detailed analysis.

(1) Epoch. It is well-known that during an epoch, the model undergoes both forward computation and backpropagation. Consequently, an increase in epochs leads to an increase in training time. To investigate the balance between epochs and accuracy, we conducted a set of experiments, the results of which are shown in Fig. 8. Evidently, for the HTTPS-D dataset, as the number of epochs increases, so does accuracy. Within 5 epochs, the accuracy reaches 0.8912, with a significant growth rate, peaking at the 21st epoch. However, after 8 epochs, the benefits of increased epochs slow down, and accuracy begins to decline after the 22nd epoch. Similarly, for the USTC-TFC2016(benign) dataset, SAT-Net achieves its best performance at the 14th epoch, with significant gains in the first 7 epochs. Therefore, in practical applications, to achieve the optimal balance between cost and performance, the number of epochs should be selected between 11 and 15.

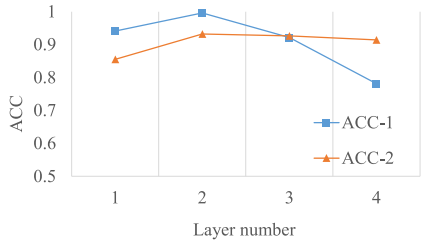
(2) Dataset Size. The size of the dataset influences the quantity and quality of the features of Packet Byte Graphs (PBGs) that the model learns. Consequently, we varied the dataset size, and Fig. 9 presents the trend in the accuracy of the SAT-Net model under different sizes of the HTTPS-D dataset. For the HTTPS-D dataset, we randomly selected 20%, 40%, 60%, and 80% of the traffic samples from the original dataset. Even when using a dataset that is only 20% of the original size, the model's accuracy remains above 90%, which suggests that the PBGs we constructed are a stable and reliable representation of encrypted traffic. As the dataset size increases, the improvement in model performance is not pronounced, indicating that SAT-Net has a certain degree of resistance to interference. The experimental results on USTC-TFC2016(benign) also demonstrate this point. Since capturing traffic data is relatively straightforward, yet subject to significant concept drift, we prefer to use a dataset that is relatively larger.



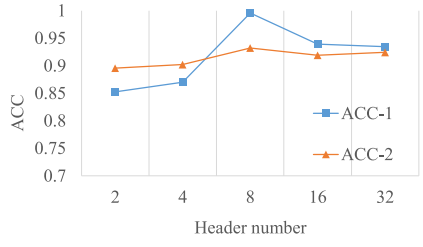
(a) Trend in Sliding Window Size Index changes affecting ACC situation.



(b) Trend in the situation of changes in learning rate affecting ACC.



(c) Trend in SAGEConv layer changes affecting the ACC situation.



(d) Trend in the number of head of attention mechanism affecting the situation in the ACC.

Fig. 7. Experimental results of sensitivity analyses on the HTTPS-D dataset.

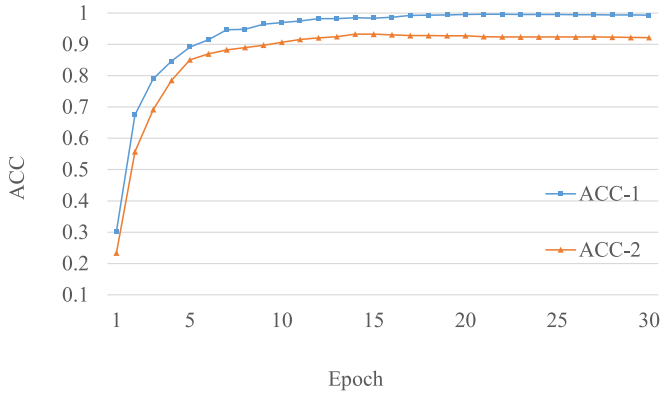


Fig. 8. Effect of training epoch on the performance of SAT-Net.

6. Limitations

The experiments conducted in Section 5 have demonstrated the advantageous properties of SAT-Net. Next, we discuss its limitations. The first issue is that our SAT-Net takes PBGs as input, but constructing

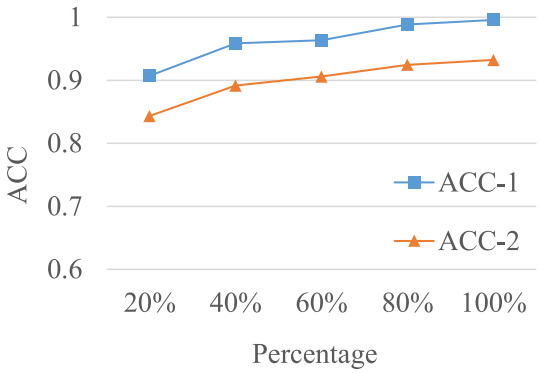


Fig. 9. Impact of dataset size on SAT-Net performance.

information-rich PBGs requires complex computations (such as setting dynamic sliding window sizes and calculating PPMI values). For instance, when dealing with video traffic (where the average number of bytes per packet is often large), constructing PBGs can be time-consuming. However, our strategy in such cases is to reduce the number of packets used per flow. Additionally, while it is relatively easy to obtain a large amount of network encrypted traffic data, most of it cannot be directly used by deep learning models after preprocessing. In future work, we could further explore using the raw bytes of unlabeled traffic flows as input for pretraining GNNs models, as this approach may contribute to a reduction in the consumption of continuous computational resources.

Moreover, the computational time required for the construction of PBGs can significantly impact the system's ability to perform in real-time. Specifically, the calculation of PPMI values and the adjustment of dynamic sliding window sizes demand considerable computational resources, particularly when dealing with large-scale traffic data, especially in a cloud computing environment. For instance, when constructing the PBG using the HTTPS-D dataset (4.3 GB), we observed an average processing time of 0.18 seconds per sample flow, with a corresponding total memory consumption of 6144 MB. To enhance real-time performance, it is advisable to consider these factors during the system design phase and to investigate potential optimization strategies, such as the implementation of more efficient algorithms or the utilization of hardware acceleration techniques.

The second issue concerns the application of SAT-Net in real-world networks. Since our SAT-Net experiments were conducted on closed datasets, when deployed in the core part of a network (such as routers), the phenomenon of concept drift in network traffic can pose challenges for SAT-Net in open-set recognition (recognizing unseen traffic patterns). It is worth noting that at the edge of the network, such as servers in data centers, traffic types are relatively stable. Therefore, in such scenarios, SAT-Net can achieve the expected results in identifying abnormal traffic types.

7. Conclusion

Considering the network's topological structure and the advantages of geometric deep learning, we propose an encrypted traffic classification method based on Staggered Attention Networks (SAT-Net). SAT-Net can utilize bidirectional flow control information and data transmission information for encrypted traffic identification. SAT-Net represents the original encrypted traffic through Packet Byte Graph (PBG), where the vertices of the graph store byte features, and the edges represent the relationships between bytes. We employ the GraphSAGE algorithm to obtain the vector representation of the PBG, and then extract the rich information of PBG through a feature remapping layer. Subsequently, we utilize the Staggered Attention layer for feature fusion, in which the multi-head attention mechanism can capture traffic flow feature information from different semantic spaces. Finally, we conduct

comprehensive experiments on four public datasets and one self-collected dataset (HPPTS-D) to evaluate the generalization and effectiveness of SAT-Net. Comparative experiments with state-of-the-art methods demonstrate that SAT-Net outperforms SOTAs on all datasets. Ablation study results indicate that both the feature remapping layer and the Staggered Attention layer contribute to performance improvement. Furthermore, we find that SAT-Net is suitable for real-world network environments in terms of the balance between cost and benefit. In future research, we aim to address concept drift by obtaining a universal representation of encrypted traffic through the study of the encrypted traffic itself.

CRediT authorship contribution statement

Zhiyuan Li: Writing – review & editing, Writing – original draft, Methodology. **Hongyi Zhao:** Investigation, Data curation. **Jingyu Zhao:** Visualization, Conceptualization. **Yuqi Jiang:** Visualization, Software. **Fanliang Bu:** Validation, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was partially supported by the Double First-Class Innovation Research Project of People's Public Security University of China: No.2023SYL08.

Data availability

Data will be made available on request.

References

Arash Habibi, L., Gerard, D., Mohammad Saiful Islam, M., Ali, A.G., 2017. Characterization of tor traffic using time based features. International Conference on Information Systems Security and Privacy, pp. 253–262. https://doi.org/10.5220/0006105602530262.

Bentian, L., Dechang, P., Yunxia, L., Lin, C., 2021. DNC: a deep neural network-based clustering-oriented network embedding algorithm. J. Netw. Comput. Appl. 173, 102854. https://doi.org/10.1016/j.jnca.2020.102854.

Bo, P., Yongquan, F., Siyuan, R., Ye, W., Qing, L., Yan, J., 2021. CGNN: traffic classification with graph neural network. CoRR, 09726. https://doi.org/10.48550/arXiv.2110.09726 abs/2110.

Chang, L., Zigang, C., Gang, X., Gaopeng, G., Siu-Ming, Y., Longtao, H., 2018. MaMPF: Encrypted Traffic Classification Based on Multi-Attribute Markov Probability Fingerprints. IEEE/ACM International Workshop on Quality of Service, pp. 1–10. https://doi.org/10.1109/iwqos.2018.8624124.

Chang, L., He, L., Gang, X., Zigang, C., Zhen, L., 2019. FS-net: a flow sequence network for encrypted traffic classification. IEEE International Conference Computer and Communications, pp. 1171–1179. https://doi.org/10.1109/infocom.2019.8737507.

Chen, Z., Cheng, G., Jiang, B., Tang, S., Guo, S., Zhou, Y., 2020. Length matters: fast Internet encrypted traffic service classification based on multi-PDU lengths. 2020 16th International Conference on Mobility, Sensing and Networking (MSN), pp. 531–538. https://doi.org/10.1109/msn50589.2020.00089.

Euclides Carlos Pinto, N., Sajjad, D., Raphael, F., Alireza, Z., Rongxing, L., Ali A, G., 2023. CICIoT2023: a real-time dataset and benchmark for large-scale attacks in IoT environment. Sensors 23 (13), 5941. https://doi.org/10.3390/s23135941, 5941.

Gerard, D., Arash Habibi, L., Mohammad Saiful Islam, M., Ali, A.G., 2016. Characterization of encrypted and VPN traffic using time-related features. International Conference on Information Systems Security and Privacy, pp. 407–414. https://doi.org/10.5220/0005740704070414.

Giuseppe, A., Domenico, C., Antonio, M., Antonio, P., 2021. DISTILLER: encrypted traffic classification via multimodal multitask deep learning. J. Netw. Comput. Appl. 183–184, 102985. https://doi.org/10.1016/j.jnca.2021.102985.

Google. HTTPS on the Web-Google Transparency Report. https://transparencyreport.google.com/https/overview(accessed 11 February 2024).

Haipeng, Y., Chong, L., Peiying, Z., Sheng, W., Chunxiao, J., Shui, Y., 2022. Identification of encrypted traffic through attention mechanism based long short term memory. IEEE Trans. Big Data 8 (1), 241–252. https://doi.org/10.1109/tbdata.2019.2940675.

Haozhen, Z., Le, Y., Xi, X., Qing, L., Francesco, M., Xiapu, L., Qixu, L., 2023. TFE-GNN: a temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. The Web Conference 16713, 2066–2075. https://doi.org/10.1145/3543507.3583227 abs/2307.

Jin-yi, C., Martin, F., Neil, I., 1992. An optimal lower bound on the number of variables for graph identification. 30th Annual Symposium on Foundations of Computer Science 12 (4), 389–410. https://doi.org/10.1109/sfcs.1989.63543.

Leslie, F.S., 2020. Packet analysis for network forensics: a comprehensive survey. Forensic Sci. Int.: Digit. Invest. 32, 200892. https://doi.org/10.1016/j.fsidi.2019.200892.

Liang, Y., Chengsheng, M., Yuan, L., 2018. Graph convolutional networks for text classification, AAAI'19/IAAI'19/EAAI'19. Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, pp. 7370–7377. https://doi.org/10.48550/arXiv.1809.05679 abs/1809.05679.

Ma, S., Liu, J., Zuo, X., 2022. Survey on graph neural network. J. Comput. Res. Dev. 59 (1), 47–80. https://doi.org/10.7544/issn1000-1239.20201055.

Meng, S., Jinpeng, Z., Liehuang, Z., Ke, X., Xiaojiang, D., 2021. Accurate decentralized application identification via encrypted traffic analysis using graph neural networks. IEEE Trans. Inf. Forensics Secur. 16, 2367–2380. https://doi.org/10.1109/tifs.2021.3050608.

Michael, F., Chris, R., Eduardo, R., Jean-Alexander, M., Klaus, H., 2014. A survey of payload-based traffic classification approaches. IEEE Commun. Surv. Tutor. 16 (2), 1135–1156. https://doi.org/10.1109/surv.2013.100613.00161.

Michael, M.B., Joan, B., Yann, L., Arthur, S., Pierre, V., 2017. Geometric deep learning: going beyond Euclidean data. IEEE Signal Process. Mag. 34 (4), 18–42. https://doi.org/10.1109/msp.2017.2693418.

Michelle, C., Lars, E., Joe, T., Magnus, W., Stuart, C., 2011. Internet Assigned Numbers Authority (IANA) Procedures for the Management of the Service Name and Transport Protocol Port Number Registry, vol. 6335. Request for Comments, pp. 1–33. https://doi.org/10.17487/rfc6335.

Minghao, J., Zhen, L., Peipei, F., Wei, C., Mingxin, C., Gang, X., Gaopeng, G., 2022. Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks. Comput. Network. 217, 109309. https://doi.org/10.1016/j.comnet.2022.109309.

Mohammad, L., Mahdi Jafari, S., Ramin Shirali Hossein, Z., Mohammdsadegh, S., 2019. Deep packet: a novel approach for encrypted traffic classification using deep learning. Comput. Res. Repos. 24 (3), 1999–2012. https://doi.org/10.1007/s00500-019-04030-2.

Peng, L., Kejiang, Y., Yishen, H., Yanying, L., Cheng-Zhong, X., 2023. A novel multimodal deep learning framework for encrypted traffic classification. IEEE/ACM Trans. Netw. 31 (3), 1369–1384. https://doi.org/10.1109/tnet.2022.3215507.

Tal, S., Yuval, S., 2021. FlowPic: a generic representation for encrypted traffic classification and applications identification. IEEE Trans. Netw. Serv. Manag. 18 (2), 1218–1232. https://doi.org/10.1109/tnsm.2021.3071441.

Tao, Y., Xingshu, C., Yi, Z., Weijing, G., Zhenhui, H., 2023. Review on the application of deep learning in network attack detection. J. Netw. Comput. Appl. 212, 1–15. https://doi.org/10.1016/j.jnca.2022.103580.

Thai-Dien, P., Thien-Lac, H., Tram, T., Tien-Dung, C., Hong Linh, T., 2021. MAppGraph - mobile-app classification on encrypted network traffic using deep graph convolution neural networks. Annual Computer Security Applications Conference, pp. 1025–1038. https://doi.org/10.1145/3485832.3485925.

Theophilus, B., Aditya, A., David, A.M., 2010. Network traffic characteristics of data centers in the wild. IMC 267–280. https://doi.org/10.1145/1879141.1879175.

Ting-Li, H., Yan, L., Tong, Z., 2021. Encrypted network traffic classification using a geometric learning model. IFIP/IEEE Symp. Integr. Netw. Manag. 376–383.

Ting-Li, H., Yan, L., Peilong, L., Tong, Z., 2023. Flow-based encrypted network traffic classification with graph neural networks. IEEE Trans. Netw. Serv. Manag. 20 (2), 1224–1237. https://doi.org/10.1109/tnsm.2022.3227500.

Tiru, W., Xi, X., Qing, L., Qixu, L., Guangwu, H., Xiapu, L., Yong, J., 2023. BehavSniffer: sniff user behaviors from the encrypted traffic by traffic burst graphs. 2023 20th Annual IEEE International Conference on Sensing, Communication, and Networking, SECON, pp. 456–464. https://doi.org/10.1109/secon58729.2023.10287511.

Tomasz, B., Valentín, C., Pere, B., 2015. Independent comparison of popular DPI tools for traffic classification. Comput. Network. 76. https://doi.org/10.1016/j.comnet.2014.11.001. C: 75.0-89.0.

Wajdi, B., Wonjun, L., 2020. Detecting malign encrypted network traffic using Perlin noise and convolutional neural network, computing and communication: 0200-0206. https://doi.org/10.1109/ccwc47524.2020.9031116.

Wei, W., Ming, Z., Jinlin, W., Xuewen, Z., Zhongzhen, Y., 2017a. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. Intelligence and Security Informatics, pp. 43–48. https://doi.org/10.1109/isi.2017.8004872.

Wei, W., Ming, Z., Xuewen, Z., Xiaozhou, Y., Yiqiang, S., 2017b. Malware traffic classification using convolutional neural network for representation learning. International Conference on Information Networking, pp. 712–717. https://doi.org/10.1109/icoin.2017.7899588.

William, L.H., Rex, Y., Jure, L., 2017. Inductive representation learning on large graphs. Adv. Neural Inf. Process. Syst. 30, 1024–1034. https://doi.org/10.48550/arXiv.1706.02216.

Xiaodong, Z., Jian, G., Maoli, W., Peng, G., Guowei, Z., 2023. IP traffic behavior characterization via semantic mining. J. Netw. Comput. Appl. 213, 103603. https://doi.org/10.2139/ssrn.4250125, 103603.

Xin, W., Shuhui, C., Jinshu, S., 2020. App-net: a hybrid neural network for encrypted mobile traffic classification. IEEE Conf. Comput. Commun. 424–429. https://doi.org/10.1109/infocomwkshps50562.2020.9162891.

Xinjie, L., Gang, X., Gaopeng, G., Zhen, L., Junzheng, S., Jing, Y., 2022. ET-BERT: a contextualized datagram representation with pre-training transformers for encrypted traffic classification. The Web Conference, pp. 633–642. https://doi.org/10.1145/3485447.3512217.

Yaxuan, Q., Lianghong, X., Baohua, Y., Yibo, X., Jun, L., 2009. Packet classification algorithms: from theory to practice. IEEE Conference on Computer Communications, p. 648. https://doi.org/10.1109/infcom.2009.5061972.

Ziqing, Z., Cuicui, K., Gang, X., Zhen, L., 2019. Deep forest with LRRS feature for fine-grained website fingerprinting with encrypted SSL/TLS. International Conference on Information and Knowledge Management, pp. 851–860. https://doi.org/10.1145/3357384.3357993.

Zonghan, W., Shirui, P., Fengwen, C., Guodong, L., Chengqi, Z., Philip S, Y., 2021. A comprehensive survey on graph neural networks. IEEE Transact. Neural Networks Learn. Syst. 32 (1), 4–24. https://doi.org/10.1109/tnnls.2020.2978386.

Zulong, D., Gaogang, X., Xin, W., Rui, R., Xuying, M., Guangxing, Z., Kun, X., Mingyu, Q., 2023. EC-GCN: a encrypted traffic classification framework based on multi-scale graph convolution networks. Comput. Network. 224, 109614. https://doi.org/10.1016/j.comnet.2023.109614, 109614.

–Zhiyuan Li received the B.S. degree in Cyber Security in Law Enforcement from Jiangxi Police College in 2021. He is currently pursuing a master's degree (in the direction of big data technology) and has been studying at the People's Public Security University of China (PPSUC) since 2022, where his research interests include network traffic analysis and deep learning.

–Hongyi Zhao obtained his Bachelor's degree in Cybersecurity and Law Enforcement from the People's Public Security University of China in 2022 and is currently pursuing a Master's degree in Security Prevention Engineering. He has been studying at the People's Public Security University of China since 2018. His research interests include network traffic analysis and natural language processing.

– Jingyu Zhao received the B.S. degree in Automation from Northeastern University (China) in 2021, and is currently pursuing the M.S. degree (in the direction of security prevention engineering), and has been studying at the People's Public Security University of China (PPSUC) since 2022, where his research interests include cybersecurity prevention and control, and information transmission.

– Yuqi Jiang received a Bachelor's degree in Cybersecurity and Law Enforcement from the People's Public Security University of China in 2021. Since 2022, she has been pursuing a Master's degree in the field of Cyberspace Security Law Enforcement Technology at the same university. Her research interests include natural language processing, imbalanced data processing, text classification, and related areas.

–Fanliang Bu is a Doctor of Engineering from the School of Electronics and Information Engineering at Xi'an Jiaotong University, and a Postdoctoral Fellow in Acoustics at the Institute of Acoustics, Chinese Academy of Sciences. He is currently a professor at the School of Information and Network Security, People's Public Security University of China, with research interests in intelligent social governance and security protection engineering.